

Deep dive in Riverpod



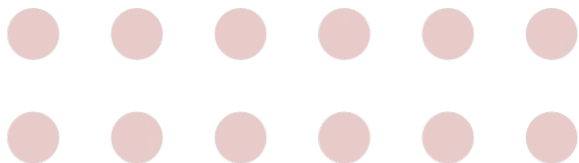
Simone Bonfrate
Dev Flutter @ Cocobuk

bonfry.com



Di cosa parleremo

- Cos'è Riverpod
- I Riverpod Provider
- Un piccolo extra



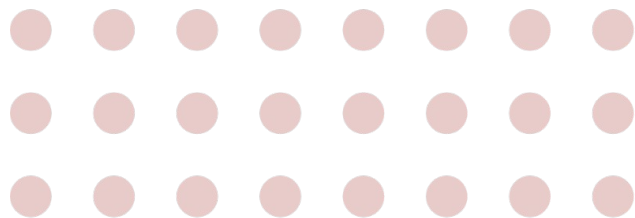
Cos'è Riverpod?

Riverpod reimplementa il concetto di Provider attraverso una definizione globale, separandolo dalla Widget Tree

```
Provider(  
  create: (_) => MyModel(),  
  child: MyApp()  
)
```

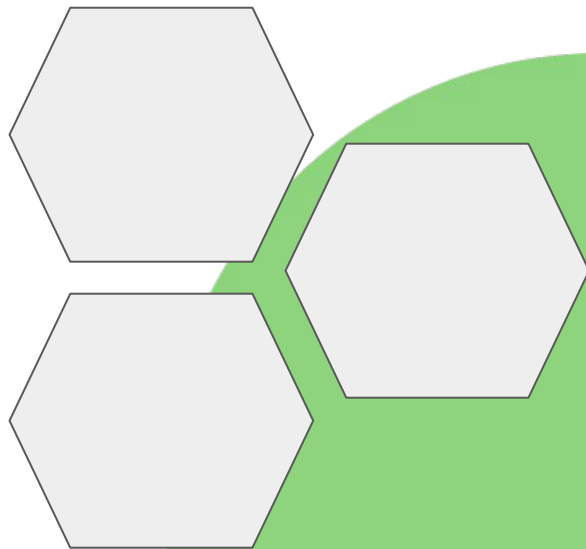


```
final myProvider = Provider((ref) {  
  return MyValue();  
});
```



Vantaggi di Riverpod

Se con l'approccio standard il provider è accessibile solo per nodi figli della Widget Tree, con Riverpod si slega dalla struttura e permette di richiamarli ovunque nel codice.



Combinazione di Provider

E' possibile combinare più provider per cambiare lo stato di un Provider



```
final cityProvider = Provider((ref) => 'London');  
  
final weatherProvider = FutureProvider((ref) async {  
  final city = ref.watch(cityProvider);  
  return fetchWeather(city: city);  
});
```



Integrazione nei Widget

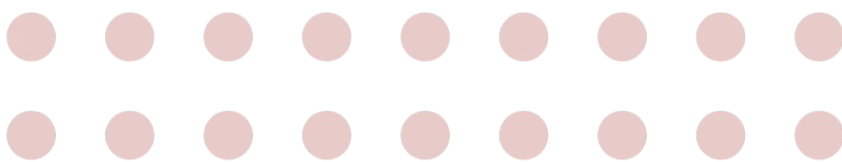
Un Widget che estendono un ConsumerWidget permettono di aggiornare la UI al cambio dello stato di un provider.

```
class HomeView extends ConsumerWidget {  
  const HomeView({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context, WidgetRef ref) {  
    // usa ref per stare in ascolto di un provider  
    final counter = ref.watch(counterProvider);  
    return Text('$counter');  
  }  
}
```

Un widget può interagire con un provider attraverso l'oggetto WidgetRef che espone 2 metodi:

- **watch(provider)**: restituisce lo stato del provider e aggiorna il widget ad ogni suo cambiamento (si usa direttamente nel builder)
- **read(provider)**: restituisce lo stato del provider una sola volta
- **listen(provider, (state){...})**: ascolta gli aggiornamenti dello stato senza dover rieseguire il build dell'interfaccia

Ottimizzare i rebuild



```
class HomeView extends ConsumerWidget {  
  const HomeView({Key? key}) : super(key: key);  
  
  Widget build(BuildContext context, WidgetRef ref) {  
    final userName = ref.watch(userProvider).name;  
  
    return const Center(  
      child: Text(userName),  
    );  
  }  
}
```

```
class HomeView extends ConsumerWidget {  
  const HomeView({Key? key}) : super(key: key);  
  
  Widget build(BuildContext context, WidgetRef ref) {  
    final userName = ref.watch(  
      userProvider.select((user) => user.name),  
    );  
  
    return const Center(  
      child: Text(userName),  
    );  
  }  
}
```

Qualora si volesse ottimizzare il numero di rebuild di un widget, piuttosto che adottare `ref.read` (manca di reattività) è possibile filtrare una particolare porzione dello stato attraverso il `provider.select()`.

Se la quel particolare sottoinsieme dello stato viene aggiornato meno frequentemente rispetto allo stato intero, il widget ribuilda meno frequentemente.

Dispose dei Provider

Essendo dichiarazioni globali e non inserite nell'albero dei Widget, risulterebbe complesso rilasciare le risorse dei provider.

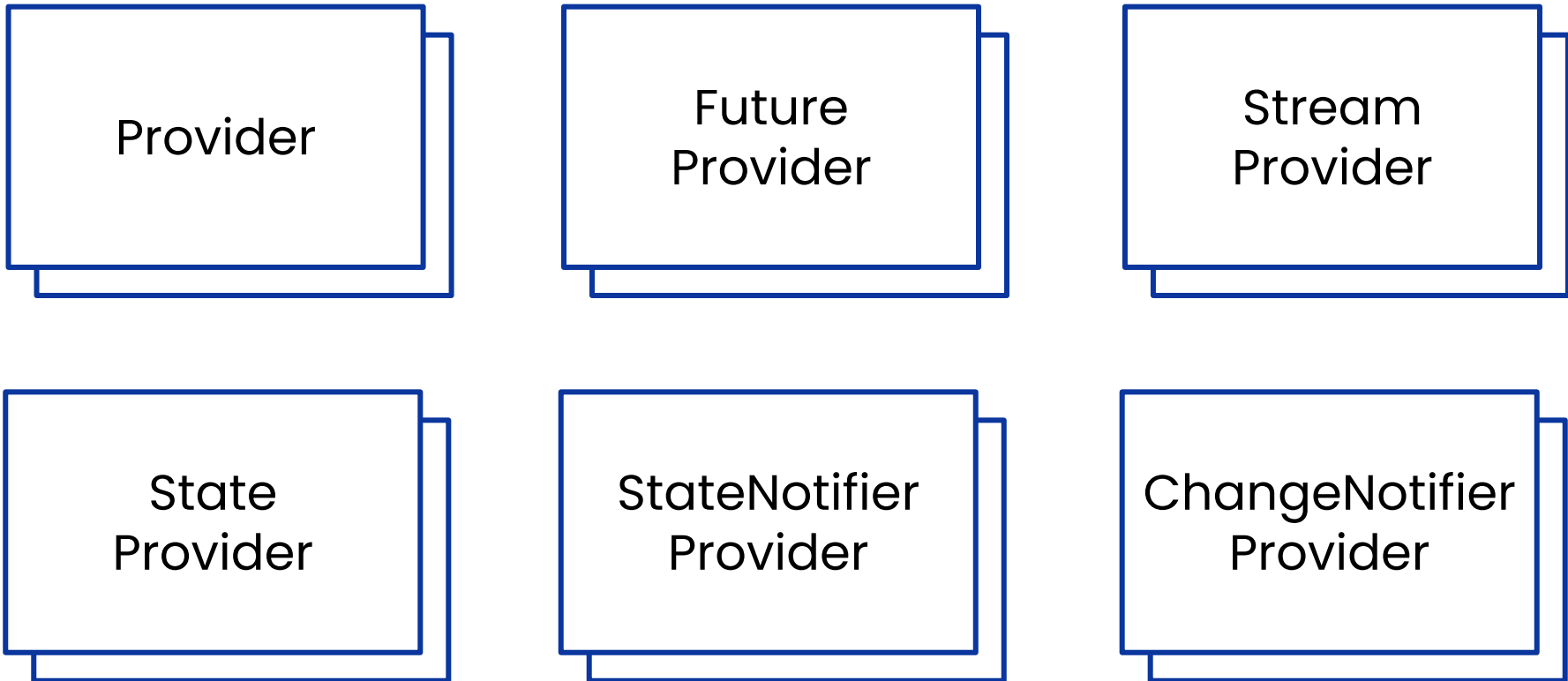
Riverpod implementa il modificatore `.autoDispose`, affinché al dispose del ConsumerWidget venga liberata la memoria occupata dallo stato del provider.



```
final userProvider = StreamProvider.autoDispose<User>((ref) { ... });
```

Al successivo richiamo, il provider ritorna allo stato iniziale.

I provider di Riverpod



Provider

Future
Provider

Stream
Provider

State
Provider

StateNotifier
Provider

ChangeNotifier
Provider

Provider

Il provider più semplice che consente di conservare un valore di sola lettura.



Tuttavia, può cambiare a seconda dell'aggiornamento dello stato di un altro provider



```
final endpointProvider = Provider<String>(  
  (ref) {  
    return 'https://flutter.festival/api'  
  },  
);
```



```
final endpointProvider = Provider<String>(  
  (ref) {  
    final deploymentType = ref.watch(deploymentProvider);  
  
    return deploymentType.isProduction  
      ? 'https://flutter.festival/api'  
      : 'https://test.flutter.festival/api';  
  },  
);
```

FutureProvider

Un provider che espone valori asincroni.

E' possibile:

- accedere direttamente al Future attraverso il read / watch del futureProvider.future
- ottenere un AsyncValue che consente di restituire uno specifico valore in base allo stato corrente attraverso il metodo .when()



```
final configProvider = FutureProvider<Configuration>((ref) async {  
  final content = json.decode(  
    await rootBundle.loadString('assets/configurations.json'),  
  ) as Map<String, Object?>;  
  
  return Configuration.fromJson(content);  
});
```

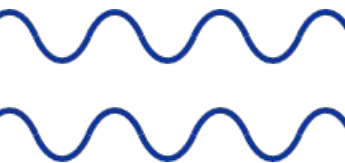


```
Widget build(BuildContext context, WidgetRef ref) {  
  AsyncValue<Configuration> config = ref.watch(configProvider);  
  
  return config.when(  
    loading: () => const CircularProgressIndicator(),  
    error: (err, stack) => Text('Error: $err'),  
    data: (config) {  
      return Text(config.host);  
    },  
  );  
}
```

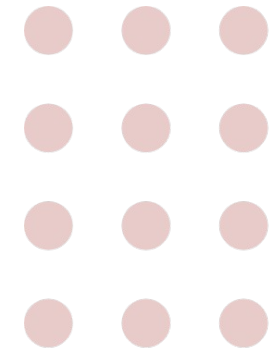
StreamProvider

E' simile al FutureProvider ma applicato agli Stream

Effettuando il read / watch del provider è possibile ottenere un AsyncValue. Applicando invece provider.stream o provider.future si potrà ottenere rispettivamente lo stream oppure il suo ultimo valore emesso



```
final messageProvider = StreamProvider<String>((ref) async* {  
  final channel = IOWebSocketChannel.connect(  
    'ws://echo.websocket.org',  
  );  
  
  ref.onDispose(() => channel.sink.close());  
  
  await for (final value in channel.stream) {  
    yield value.toString();  
  }  
});
```



ChangeNotifierProvider

Provider che espone i ChangeNotifier, un componente implementato direttamente in Flutter per centralizzare la logica ed aggiornare chi è in ascolto dei cambi di stato.

Creato per agevolare la migrazione verso Riverpod, tuttavia, l'adozione rappresenta una bad practice perché capace di gestire stati mutabili che possono rompere alcune funzionalità del framework (come il .select).

```
class CounterNotifier extends ChangeNotifier {
  int counter = 0;

  void increase() {
    counter++;
    notifyListeners();
  }

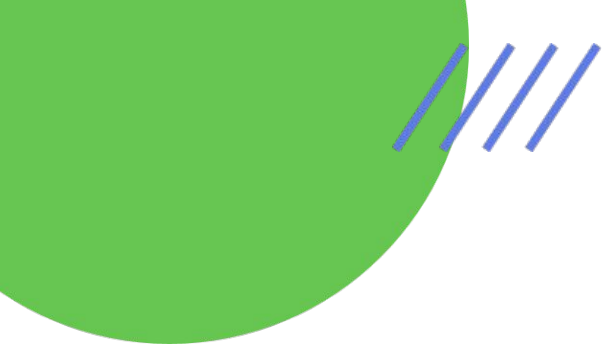
  void decrease() {
    counter--;
    notifyListeners();
  }
}

final counterProvider = ChangeNotifierProvider<CounterNotifier>(
  (ref) => CounterNotifier(),
);

...

@override
Widget build(BuildContext context, WidgetRef ref) {
  // rebuild the widget when the todo list changes
  final counter = ref.watch(counterProvider).counter;

  // Let's render the todos in a scrollable list view
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(
        counter.toString(),
        textAlign: TextAlign.center,
      ),
      TextButton(
        child: Text('+'),
        onPressed: () => ref.read(counterProvider.notifier).increase(),
      ),
      TextButton(
        child: Text('-'),
        onPressed: () => ref.read(counterProvider.notifier).decrease(),
      ),
    ],
  );
}
```



StateNotifierProvider

Rappresenta l'approccio corretto per centralizzare la logica con Riverpod. Lo StateNotifier espone uno stato immutabile e consente di riconoscere i suoi cambiamenti.



```
class CounterNotifier extends StateNotifier<int> {
  CounterNotifier() : super(0);

  void increase() => state = state + 1;
  void decrease() => state = state - 1;
}

final counterProvider = StateNotifierProvider<CounterNotifier, int>(
  (ref) => CounterNotifier(),
);

@override
Widget build(BuildContext context, WidgetRef ref) {
  // rebuild the widget when the todo list changes
  final counter = ref.watch(counterProvider);

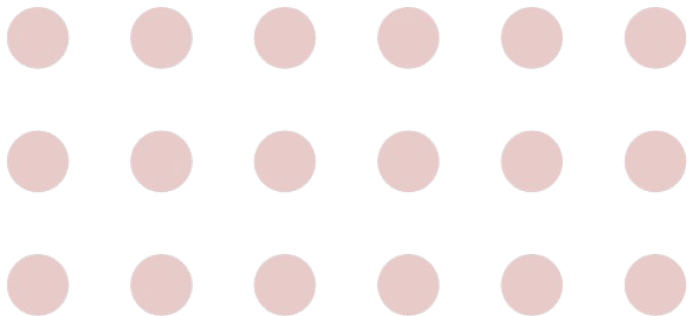
  // Let's render the todos in a scrollable list view
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(
        counter.toString(),
        textAlign: TextAlign.center,
      ),
      TextButton(
        child: const Text('+'),
        onPressed: () => ref.read(counterProvider.notifier).increase(),
      ),
      TextButton(
        child: const Text('-'),
        onPressed: () => ref.read(counterProvider.notifier).decrease(),
      ),
    ],
  );
}
```



StateProvider

Una semplificazione dello
StateNotifier, adatto a gestire dati
atomici (come interi ed enum).

Si differenzia per la possibilità di
cambiare cambiare lo stato
direttamente nel widget



```
final counterProvider = StateProvider<int>((ref) => 0);

@override
Widget build(BuildContext context, WidgetRef ref) {
  // rebuild the widget when the todo list changes
  final counter = ref.watch(counterProvider);

  // Let's render the todos in a scrollable list view
  return Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: <Widget>[
      Text(
        counter.toString(),
        textAlign: TextAlign.center,
      ),
      TextButton(
        child: const Text('Reset'),
        onPressed: () {
          //Approccio 1
          ref.read(counterProvider.notifier).state = 0;
        },
      ),
      TextButton(
        child: const Text('+'),
        onPressed: () {
          //Approccio 2
          ref.read(counterProvider.notifier).update((state) => state + 1);
        },
      ),
      TextButton(
        child: const Text('-'),
        onPressed: () {
          //Approccio 2
          ref.read(counterProvider.notifier).update((state) => state - 1);
        },
      ),
    ],
  );
}
```



One More Thing

Riverpod essendo di base un provider, può essere esteso anche con altri State Management esistenti.

Come ad esempio BLoC



Riverbloc

Esempio



```
@immutable
class CounterState {
  final int counter;

  const CounterState(this.counter);
}

@immutable
class CounterEvent {}

class CounterIncreased extends CounterEvent {}

class CounterDecreased extends CounterEvent {}

class CounterBloc extends Bloc<CounterEvent, CounterState> {
  CounterBloc() : super(const CounterState(0)) {

    on<CounterIncreased>((event, emit) => emit(CounterState(state.counter + 1)),
    );

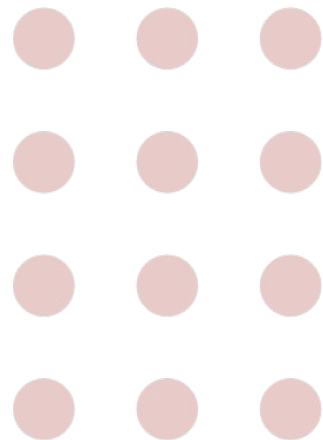
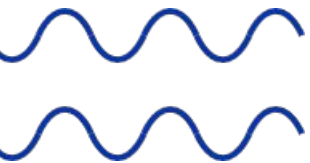
    on<CounterDecreased>((event, emit) => emit(CounterState(state.counter - 1)),
    );
  }
}

final counterProvider = BlocProvider<CounterBloc, CounterState>(
  (ref) => CounterBloc(),
);
```



```
class Counter extends ConsumerWidget {
  const Counter({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context, WidgetRef ref) {
    CounterState counterState = ref.watch(counterProvider);
    return Row(
      mainAxisAlignment: MainAxisAlignment.min,
      children: <Widget>[
        IconButton(
          onPressed: () => ref.read(counterProvider.notifier).add(
            CounterIncreased(),
          ),
          icon: const Icon(Icons.add),
        ),
        SizedBox(
          height: 50,
          width: 50,
          child: Center(
            child: Text(counterState.counter.toString()),
          ),
        ),
        IconButton(
          onPressed: () => ref.read(counterProvider.notifier).add(
            CounterDecreased(),
          ),
          icon: const Icon(Icons.remove),
        ),
      ],
    );
  }
}
```

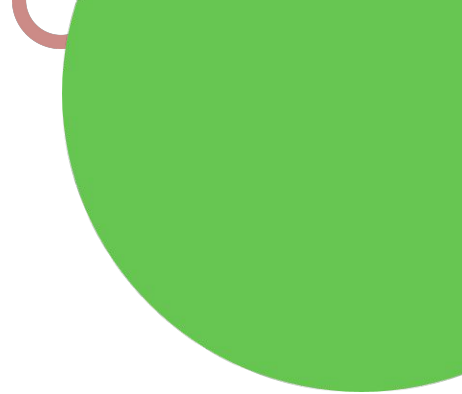
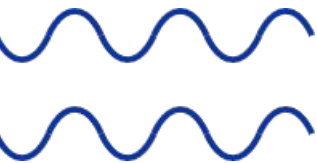


Let's coding



Scansiona il QR code per
accedere agli esempi





Grazie per l'attenzione

